

# PSP Database Archiving & Purging

- Problem Statement
- Goal
- Impact
- Solutions
  - Data Selection
  - Data Analysis
    - Targeted Size Reduction
    - Final Target companies for archive
    - Breakdown of Targeted companies
    - Stale companies
  - Proposed Solution
- Design
  - Design Principal
    - Data Integrity -
    - Business Continuity
    - Scalability
    - Ease-of-Use
    - Interoperability
  - Component Diagram
    - Library
      - Open source Plan
      - Wrapper for PSP DB archival
    - Web-Hook with EFS
  - Sequence Diagram
    - Archive Company State
  - Execution plan
  - Design Decision
    - Archive in DB
    - Why PostgresQL
    - Why Web-hook
      - Other solutions
    - Data validation
    - Transaction management with Chain Transaction Management
      - Two-phase commit Pattern
        - Support in Java spring boot
        - Problem faced:
      - Chain Transaction Management
    - Why one DB in pre-prod and Staging
    - Pause on Data Archival Track
      - Not much ROI because of below:
      - Use-cases:
      - Prerequisites/Considerations
    - Rollback Strategy
  - Clients of PSP DB
- References

## Problem Statement

The current size of the Main PSP Database is a whopping ~38TB, with continuous growth at an alarming rate of approximately ~4TB per year. This excessive size not only hampers the system's efficiency and performance, but also results in increased expenses related to storage and maintenance. Consequently, it leads to substantial operational inconvenience, impacting our vital business operations.

Given the size, the database has been exhibiting slower data retrieval, escalating costs for both storage and maintenance and proving to be a bottleneck in the processes. Moreover, the data redundancy coupled with an huge data volume increases the risk associated with data management, thereby affecting overall system reliability. The financial implications are critical as well, the continuous increase in storage costs are proving detrimental to the company's financial health.

The reduction in the database size will substantially aid in this initiative, potentially making the transition smoother by reducing the time and resources required for data migration.

Additionally, the increasing size of the database is posing a challenge to our company's compliance with data regulation policies. By reducing the database size through the Data Archival initiative, we can improve compliance by ensuring easier management and improved security of data. Effective data archiving will remove obsolete data from the main database, reducing the information load while ensuring all necessary historical data is well-preserved, conforming to both industry regulations and internal data policies.

## Goal

- **Goal:** Decrease the database size by ~20% by archiving redundant data

To accomplish this, we'll try the following strategies:

- Eliminate companies from the database that have not received a validation call within the past **decade** and are considered inactive.

If the database size has not been reduced by the required 20% after the first step, we will proceed to the following strategy:

- In compliance with our data retention policies, we can limit our database to information from the past **seven years** only.

If the database is still larger than desired after applying the second strategy, we will carry out a third one:

- Purge the records of those companies for which entitlements are deactivated and have not received any validation call within the last **three years**.

## Impact

1. **Cost Optimisation:** The most immediate positive impact of reducing the database size by approximately 20% will be cost optimisation. With less redundant data to store and manage, we will be able to significantly reduce our database costs.
2. **Enhanced Processing Speed:** Presently, despite indexing, our Postgres database is taking more time with queries due to a large amount of data. By archiving and thus eliminating the unnecessary data, we are likely to improve processing speed, making queries more efficient and significantly reducing query response times.
3. **Improved Data Computation:** Currently, our databases hold roughly 50 TB of data, making it challenging to analyze the precise amount of data that needs maintaining. However, data archiving will facilitate not only the computation of critical data but also effective data management. With less data to sift through, the process of identifying and accessing critical data will be expedited, improving overall operational efficiency.
4. **Streamlined Data Management:** The less clutter there is in our databases, the easier it is to manage. Archiving allows for better organization of resources and makes database management more streamlined.
5. **Enhanced Security:** With fewer data to manage, it's easier to secure our databases. By storing only what's necessary, we reduce the risk of data breaches.
6. **Regulatory Compliance:** Depending on the region and industry, there can be specific requirements for how long certain types of data need to be retained. Archiving will ensure that we are adhering to these guidelines, ensuring compliance while also managing our resources efficiently.

## Solutions

### Data Selection

**Target Reduction in Size to be achieved:** ~20%

The selection criteria for the Data Archival initiative have been established based on a strategic analysis of the PSP database, aimed at identifying stagnant and redundant data. A primary focus of this analysis has been on data sets from two main sources: **IOP and QBOP**. These data sets were selected due to their redundancy and non-growth since 2018, making them ideal candidates for archival.

In addition to the IOP and QBOP data, we have also included data pertaining to **Inactive companies** within our selection criteria. The term 'inactive companies' refers specifically to companies that have exhibited no activity within the last three years. This entails companies with deactivated entitlements and those, of which, validation calls have not been made.

**Phase 1:** We will commence with the archiving of data from the Primary database.

### Data Analysis

#### Targeted Size Reduction

Complete analysis details : [PSP Data Archival Analysis](#)

Approximation of size reduction based on the analysis. These calculation are based on the only tables with size more than 1 GB.

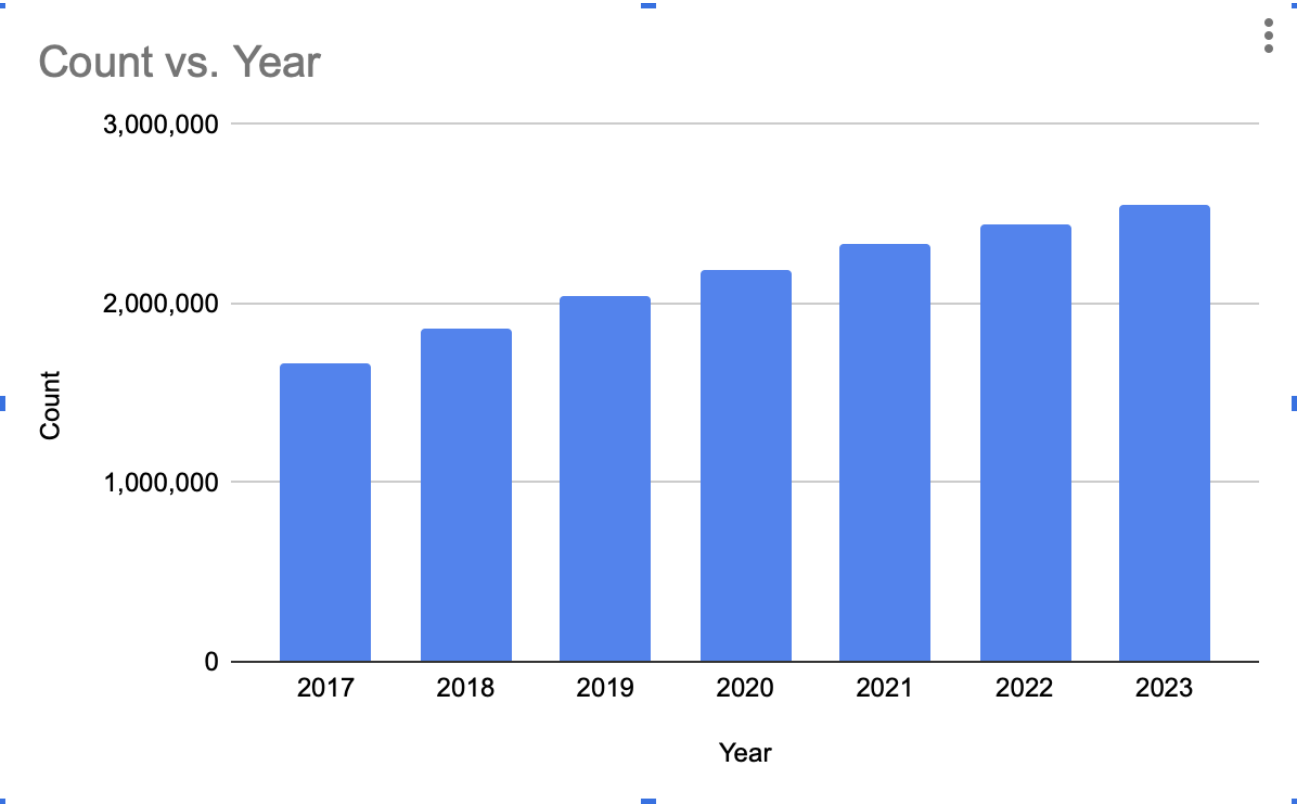
| Source               | Size reduction (In GB) till 2017 | Size reduction (In GB) till 2021 |
|----------------------|----------------------------------|----------------------------------|
| QBDT                 | 3721                             | TBC                              |
| IOP                  | 1035                             | 1035                             |
| QBOP                 | 21                               | 21                               |
| Total                | 4777                             |                                  |
| % total size (26 TB) | ~ 12%                            |                                  |

#### Final Target companies for archive

| Companies | Count till 2017 | Count till 2021 | Remarks       |
|-----------|-----------------|-----------------|---------------|
| QBDT      | 1.66 M          | 2.32M           |               |
| IOP       | 628 K           | 628 K           | All companies |
| QBOE      | 31 K            | 31 K            | All companies |

Breakdown of Targeted companies

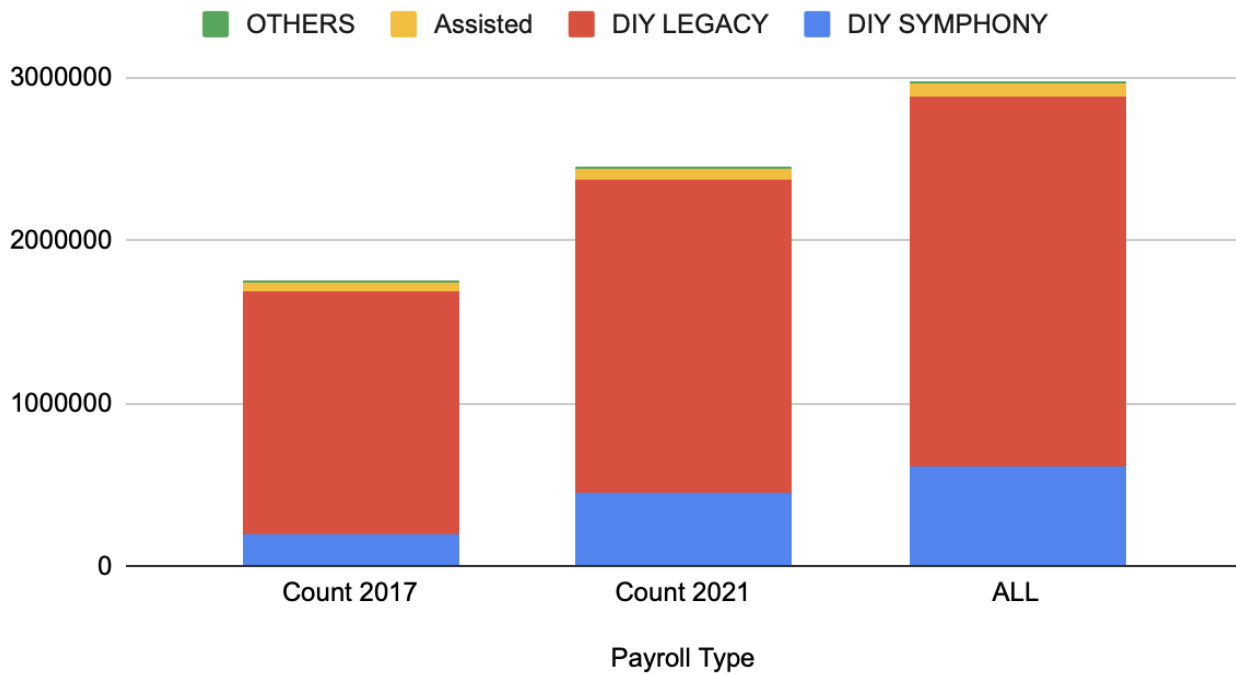
Inactive Entitlement with last validation:



| Year  | 2017      | 2018      | 2019      | 2020      | 2021      | 2022      | 2023      |
|-------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| Count | 1,668,876 | 1,855,978 | 2,041,660 | 2,190,983 | 2,328,027 | 2,446,147 | 2,552,977 |

Based on Payroll Type:

## Count 2017, 2021 and ALL



| Payroll Type | Count last validation 2017 | Count last validation 2021 | ALL      |
|--------------|----------------------------|----------------------------|----------|
| DIY SYMPHONY | 198887                     | 452540                     | 606174   |
| DIY LEGACY   | 1489375                    | 1914783                    | 2275951  |
| Assisted     | 47620                      | 70264                      | 81476    |
| OTHERS       | 15210                      | 15256                      | 16307    |
|              | ~1751092                   | ~2452843                   | ~2979908 |

### Based on Payroll services

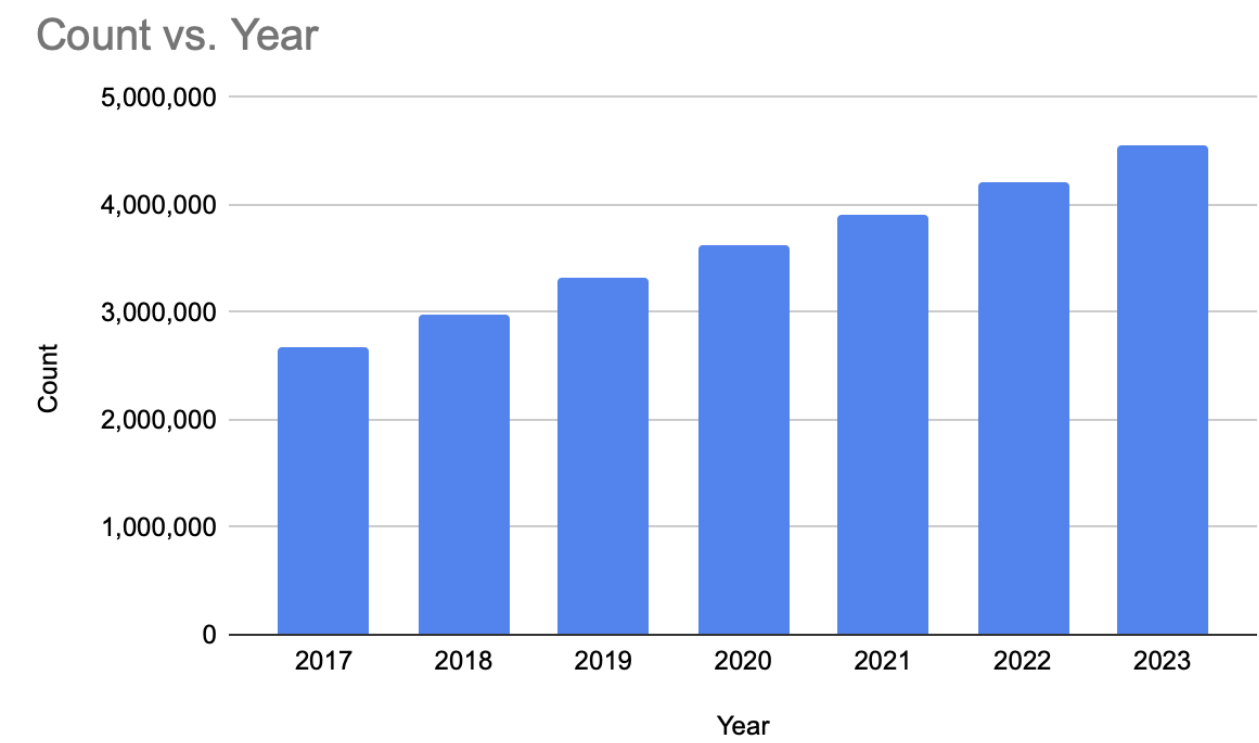
| Services                                                           | 2017    | 2021    | ALL     |
|--------------------------------------------------------------------|---------|---------|---------|
| BillPayment,Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck         | 891     | 2236    | 2778    |
| BillPayment,Cloud,CloudV2,DirectDeposit,ViewMyPaycheck             | 5345    | 11666   | 20349   |
| BillPayment,Cloud,CloudV2,DirectDeposit,ViewMyPaycheck,WorkersComp |         |         | 581     |
| BillPayment,Cloud,CloudV2,DirectDeposit,WorkersComp                |         |         | 416     |
| BillPayment,Cloud,DirectDeposit                                    | 16663   | 31943   | 39426   |
| BillPayment,Cloud,DirectDeposit,Tax                                | 1170    | 2207    | 2367    |
| Cloud                                                              | 1412567 | 1903343 | 2309095 |
| Cloud,CloudV2,CloudV3,DirectDeposit,ViewMyPaycheck                 |         |         | 425     |
| Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck                     | 4999    | 11158   | 14728   |
| Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck,WorkersComp         |         | 755     | 905     |
| Cloud,CloudV2,DirectDeposit,Tax,WorkersComp                        | 688     | 1301    | 1363    |
| Cloud,CloudV2,DirectDeposit,ViewMyPaycheck                         | 13992   | 31562   | 63646   |
| Cloud,CloudV2,DirectDeposit,ViewMyPaycheck,WorkersComp             | 405     | 1000    | 1522    |
| Cloud,CloudV2,DirectDeposit,WorkersComp                            | 824     | 1892    | 2342    |
| Cloud,CloudV2,ViewMyPaycheck                                       | 4712    | 11658   | 17230   |
| Cloud,CloudV2,WorkersComp                                          | 870     | 1966    | 2518    |

|                         |          |          |          |
|-------------------------|----------|----------|----------|
| Cloud,DirectDeposit     | 166955   | 263741   | 314669   |
| Cloud,DirectDeposit,Tax | 33317    | 45219    | 47687    |
| Cloud,Tax               | 3432     | 3458     | 3501     |
| DirectDeposit           | 531      | 574      | 629      |
| RiskAssessment          |          |          | 671      |
|                         | ~1667361 | ~2325679 | ~2846848 |

Stale companies

Stale company - A company which have its subscription enable but there is not activity (Payroll run or even validation call to validate subscription)

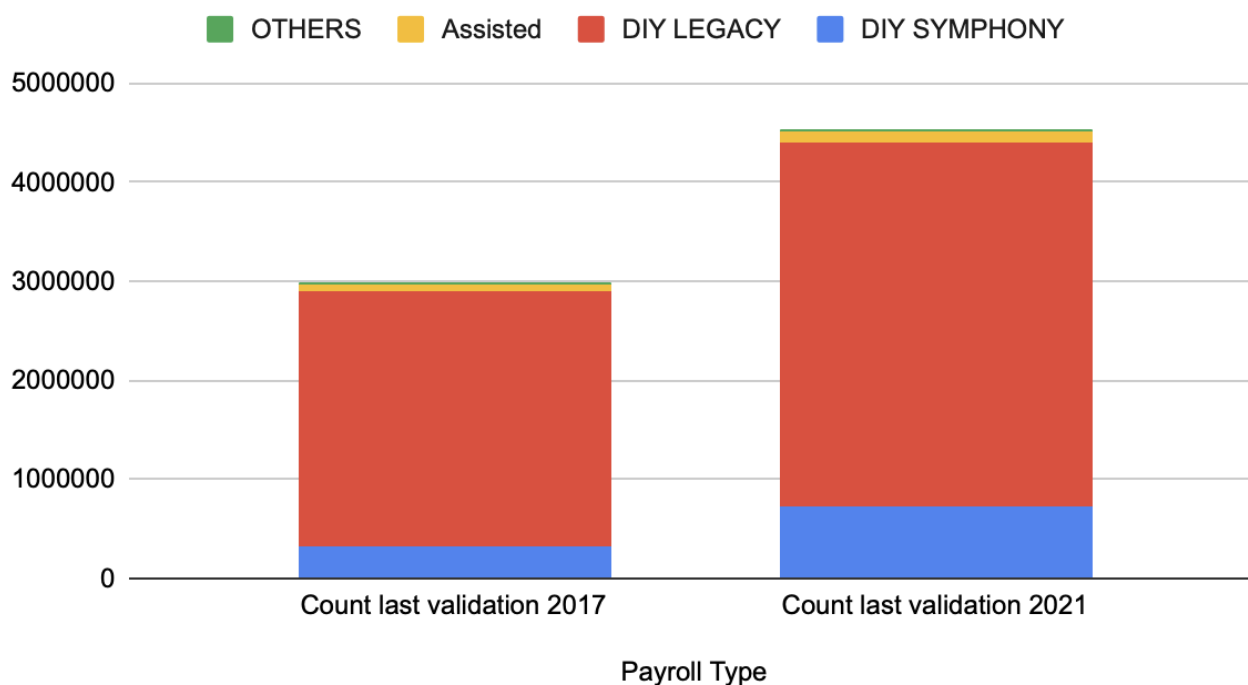
By Years



| Year  | 2017      | 2018      | 2019      | 2020      | 2021      | 2022      | 2023      |
|-------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| Count | 2,666,580 | 2,986,396 | 3,323,338 | 3,617,489 | 3,907,767 | 4,206,345 | 4,556,372 |

Based on Payroll Type:

## Count 2017 and 2021



| Payroll Type | Count last validation 2017 | Count last validation 2021 |
|--------------|----------------------------|----------------------------|
| DIY SYMPHONY | 314,580                    | 733,974                    |
| DIY LEGACY   | 2,584,624                  | 3,675,244                  |
| Assisted     | 62,980                     | 108,599                    |
| OTHERS       | 20,585                     | 20,705                     |
|              | ~2,982,769                 | ~4,538,522                 |

### Based on Payroll services

| Services                                                           | 2017    | 2021    |
|--------------------------------------------------------------------|---------|---------|
| BillPayment,Cloud,CloudV2,CloudV3,DirectDeposit,ViewMyPaycheck     | 1419    | 3291    |
| BillPayment,Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck         | 1983    | 6019    |
| BillPayment,Cloud,CloudV2,DirectDeposit,ViewMyPaycheck             | 20279   | 45316   |
| BillPayment,Cloud,CloudV2,DirectDeposit,ViewMyPaycheck,WorkersComp |         | 860     |
| BillPayment,Cloud,CloudV3,DirectDeposit                            | 1279    | 2927    |
| BillPayment,Cloud,DirectDeposit                                    | 26090   | 55053   |
| BillPayment,Cloud,DirectDeposit,Tax                                | 1628    | 3479    |
| Cloud                                                              | 2396291 | 3444660 |
| Cloud,CloudV2,CloudV3,DirectDeposit,ViewMyPaycheck                 | 3228    | 7513    |
| Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck                     | 9039    | 25934   |
| Cloud,CloudV2,DirectDeposit,Tax,ViewMyPaycheck,WorkersComp         |         | 1473    |
| Cloud,CloudV2,DirectDeposit,Tax,WorkersComp                        | 875     | 1859    |
| Cloud,CloudV2,DirectDeposit,ViewMyPaycheck                         | 49939   | 117338  |

|                                                        |          |          |
|--------------------------------------------------------|----------|----------|
| Cloud,CloudV2,DirectDeposit,ViewMyPaycheck,WorkersComp | 853      | 2047     |
| Cloud,CloudV2,DirectDeposit,WorkersComp                | 1079     | 2547     |
| Cloud,CloudV2,ViewMyPaycheck                           | 10268    | 26446    |
| Cloud,CloudV2,WorkersComp                              | 1345     | 3111     |
| Cloud,CloudV3,DirectDeposit                            | 3688     | 8464     |
| Cloud,DirectDeposit                                    | 229305   | 391364   |
| Cloud,DirectDeposit,Tax                                | 39987    | 58397    |
| Cloud,Tax                                              | 3537     | 3570     |
| DirectDeposit                                          | 731      | 797      |
|                                                        | ~2802843 | ~4212465 |

## Proposed Solution

We will create a replica (Db will have only Schema not data) of our DB. Then we are going to archive a company data (From each table by table 250+) to another DB. With this we will reduce the size of the source DB and also have the Data in archive DB to rollback at any point in time.

Once the company is persisted in Archive DB for a certain period (TBD) of time we will purge that company from Archive DB.

## Design

### Design Principal

#### Data Integrity -

- Each column and row is validated before archiving to Archive DB.
- In 2 transaction commit we are validating the data 2 times.
- All the triggers are disabled so they don't change the time stamp and version of the rows.

#### Business Continuity

- At any point if there is a failure the business should not get impacted.
- The complete company copy or delete are in one transaction so any failure should revert back.

#### Scalability

- We need to migrate 3.2 M company and with all the validation and state management.
- To migrate such a huge number we need a scalable solution. We are using the WebHook (Sensors Kubernetes) so we can scale the no of pods to migrate the companies.
- To hit the WebHook (Sensors Kubernetes) multiple times we are using batch job to triggers and files as an input.

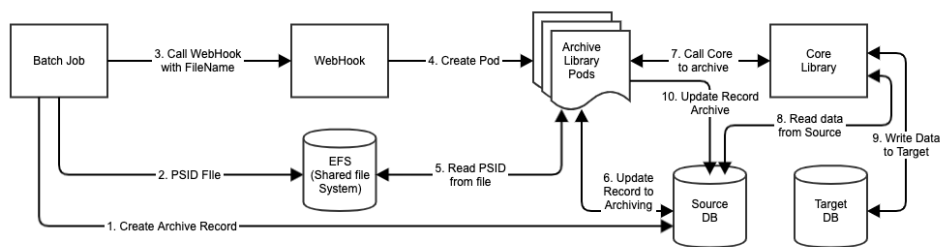
#### Ease-of-Use

- The completed solution is designed with reusability in mind.
- The complete library can be configured and used in any spring project.

#### Interoperability

- The library can be used with any source and target data bases.
- If the data type is different between source and target we can configure the convertor.

## Component Diagram



## Library

We have analysed multiple data migration tools and services but we didn't find anything which can migrate entity (logical grouping of data) to another DB.

### Open source Plan

Coz there is not Entity level migration

### Wrapper for PSP DB archival

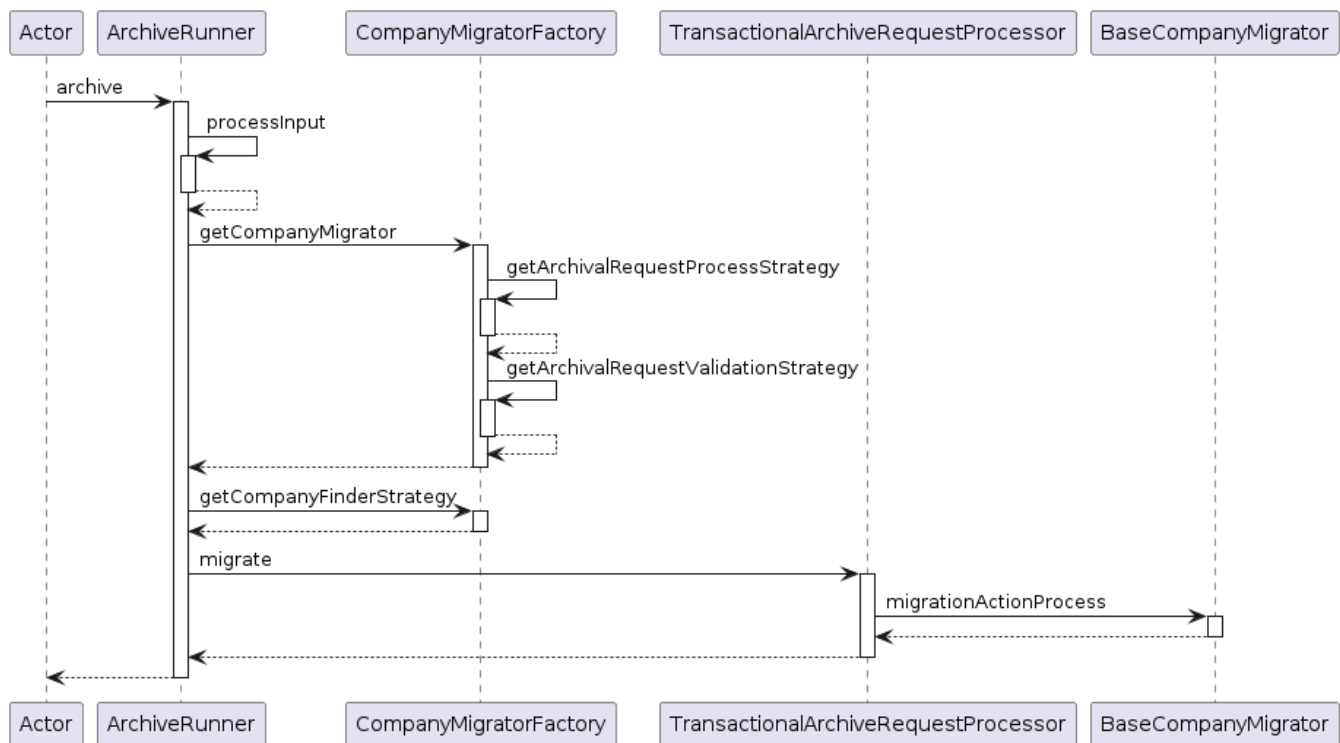
We have adopted and implemented the DB-Archival library for PSP DB Archival. The PSP DB Archival solution has following main features:

- State Management
- Multiple input methods to provide Company IDs for archival
- 2 stage transaction management using state management
- Argo events Webhook triggered execution pods using batch job.
- Validation of data during copy as well as deletion from source DB.

### Web-Hook with EFS

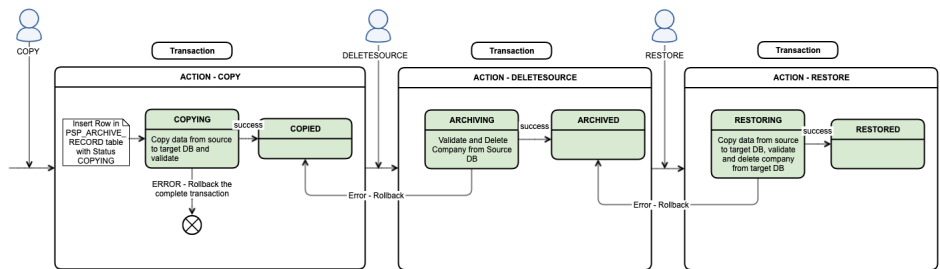
We will creating the web-hook with the EFS storage. The Web hook will trigger when the argo event is triggered. To trigger the argo event which will run the PSP DB Archival library using the arguments in the Argo event request.

## Sequence Diagram



### Archive Company State

State management has been implemented to be able to figure out what is the archival status of a company file. Possible statuses are:



FETCHED - CompanyIds have been fetched from given input source)  
 INPUT\_FILE - Company file has been added to input file as source of Archival Requests  
 COPYING - Company is being copied to target DB  
 COPIED - Company has been copied to target DB but not deleted from source DB  
 ARCHIVING - Company is being archived  
 ARCHIVED - Company has been copied to target DB and deleted from source DB  
 RESTORE -  
 RESTORING - Company is being restored to source DB  
 RESTORED - Company has been restored to source DB

ROLLED - Company archival process has been rolled back due to some issue while copying

### Execution plan

|                            |                    |                     |                        |               |
|----------------------------|--------------------|---------------------|------------------------|---------------|
| Dev Complete<br>(15th Feb) | Phase 1 (29th Feb) | Phase 2 (5th March) | Phase 3 (5-25th March) | Phase 4 (TBD) |
|----------------------------|--------------------|---------------------|------------------------|---------------|

|                                                                                                                                                                                                                                                                                                                           |                                                                                                                 |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Open source Library - 31st Jan</li> <li>Wrapper for the PSP - 31st Jan</li> <li>Library With web-hook integration - 31st Jan</li> <li>Impacted Area - 25th Jan</li> <li>Impacted Area Solution Designe - 31th Jan</li> <li>Impacted Area Completion - 15th Feb-21st Feb</li> </ul> | Archiving non-QBDT companies <ul style="list-style-type: none"> <li>Archiving IOP companies data 10%</li> </ul> | Archiving non-QBDT companies <ul style="list-style-type: none"> <li>Archiving IOP companies data 100%</li> <li>Archiving QBOE companies data</li> <li>Restoring companies to main database when needed</li> </ul> | Moving Inactive companies <ul style="list-style-type: none"> <li>Archiving Inactive companies of more than 7 years old to an archive DB data storage.</li> <li>SAP Search for the Archive company (5th March)</li> <li>Restore companies to the main database from archived storage on demand (5th March)</li> </ul> | Moving Stale companies <ul style="list-style-type: none"> <li>Archiving Active but stale companies</li> <li>Keeping latest of 3-4 years of data in main database</li> <li>No purging of these companies</li> <li>Purging companies from Main database and/or Archived database based on the retention policy.</li> </ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Design Decision

### Archive in DB

| S. No. | Approaches                          | Details                                                                                                                                                                                                                                                                                                                                      | Pros                                                                                                                        | Cons                                                                                                                         | Remarks |
|--------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------|
| 1.     | Existing solutions for DB migration |                                                                                                                                                                                                                                                                                                                                              |                                                                                                                             | No logical grouping of Data                                                                                                  |         |
| 2.     | Archived DB as another Postgres DB  | <ul style="list-style-type: none"> <li>A new archived AWS Postgres DB needed with same DB Schema</li> <li>Copy all of a company data into archived DB</li> <li>Validate company</li> <li>Remove all data of a company from main database.</li> <li>Restore: If required, copying all of a company data back to the main database.</li> </ul> | <ul style="list-style-type: none"> <li>Overall Less effort</li> <li>Easy to restore back and atomicity achieved.</li> </ul> | <ul style="list-style-type: none"> <li>Costly</li> </ul>                                                                     |         |
| 3.     | Archived DB as file system (ex: S3) | <ul style="list-style-type: none"> <li>Extract out all the data and store as INSERT statement into the AWS S3 file storage.</li> <li>Need to maintain files in such hierarchy as Company/Tables etc.</li> <li>Restoring company back to main database by running INSERT statements from file system.</li> </ul>                              | <ul style="list-style-type: none"> <li>Cheaper</li> </ul>                                                                   | <ul style="list-style-type: none"> <li>Restore back any company would be complex, difficult to achieve atomicity.</li> </ul> |         |

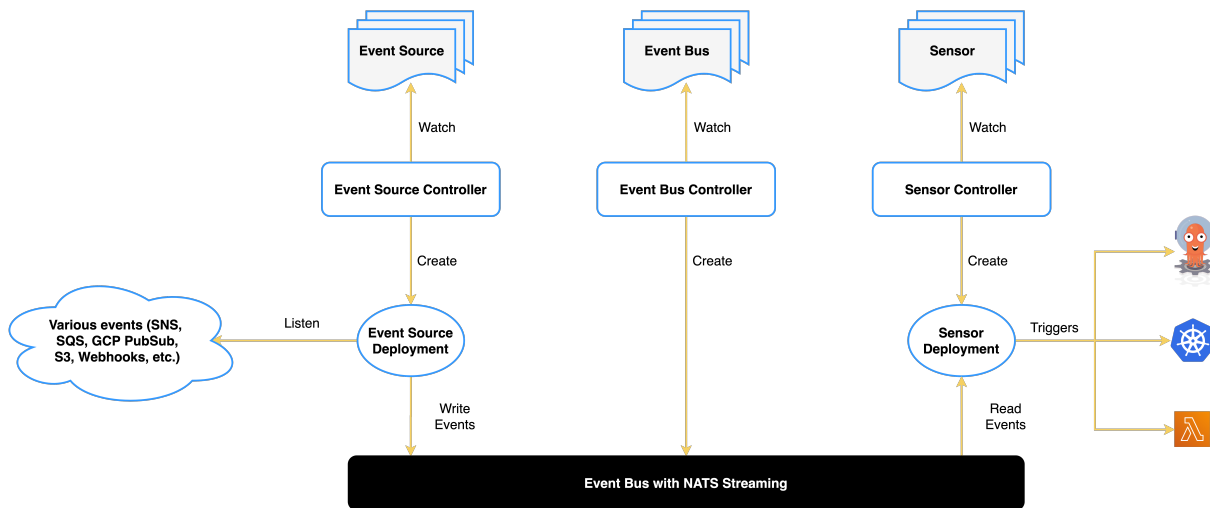
### Why PostgresQL

The 2 designs pattern data integrity and business continuity are scored very high in Postgres DB.

- To maintain the data integrity it's easy to compare data between both the DB.
- In case of the any issue its easy to restore the data from the archive postures DB.

### Why Web-hook

**Argo Events** is an event-driven workflow automation framework for Kubernetes which helps you trigger K8s objects, Argo Workflows, Serverless workloads, etc. on events from a variety of sources like webhooks, S3, schedules, messaging queues, gcp pubsub, sns, sqs, etc. For a detailed description of argo events see [this link](#). Argo-events architecture is shown below:



An **EventSource** defines the configurations required to consume events from external event sources like AWS SNS, SQS, GCP PubSub, Webhooks. It further transforms the events into the [cloudevents](#) and dispatches them over to the **eventbus**.

**Sensor** defines a set of event dependencies (inputs) and triggers (outputs). It listens to events on the eventbus and acts as an event dependency manager to resolve and execute the triggers.

A **Trigger** is the resource/workload executed by the sensor once the event dependencies are resolved.

#### Why we have used argo workflows?

We have used argo workflows to perform parallel execution of DB Archival of multiple companies which can scale depending upon on companies to be archived.

#### Other solutions

We did POCs on using CronJob pod (rejected as this solution was not scalable) and batch job directly performing DB Archival (Don't want to add load to the existing pods).

#### Data validation

- Data Validation is done using the code while copying and deleting the data.
- Validation is done via comparing number of records in each tables also each column data in each record.

### Transaction management with Chain Transaction Management

During Data Archival of a company, a transaction would mean reading data from source DB, inserting data in target DB, validating the data in source and target DB and deleting data from source DB. Since here we are dealing with distributed databases, we cannot guarantee atomicity of a transaction. It becomes responsibility of the application to make a transaction atomic.

#### Why atomicity is important?

In a database system, atomicity means that in a transaction either **all steps complete** or **no steps complete**. We have to execute queries on two separate databases to complete a transaction. If either of the queries fails, it will result in an inconsistent state. We want to prevent such an inconsistent state. We have to ensure that either the transaction completes successfully or fails. We don't want to leave the transaction midway in an inconsistent state.

Available patterns for achieving atomicity for distributed platforms:

- Two-phase commit pattern (XA transaction)
- Chain Transaction Management

#### Two-phase commit Pattern

2pc has two phases: A *prepare phase* and a *commit phase*. In the prepare phase, all microservices will be asked to prepare for some data change that could be done atomically. Once all micro-services are prepared, the commit phase will ask all the microservices to make the actual changes.

#### Support in Java spring boot

JTA provides interfaces in Java for building applications with distributed transactions. Atomikos is a lightweight transaction manager for Java that enables applications using distributed transactions to be self-contained. Essentially, our application doesn't need to rely on a heavyweight component like an application server for transactions. Atomikos is built primarily as a JTA transaction manager and, hence, implements XA architecture with a two-phase commit protocol.

Prepared Transactions are the basis of 2-phase commit pattern. A prepared statement is a parameterised and reusable SQL query which forces the developer to write the SQL command and the user-provided data separately.

### Problem faced:

While implementing a Atomikos Transaction manager to enable 2-phase commit pattern, we found that in Postgres DB, usage of prepared transactions is disabled by default. There is a server parameter **max\_prepared\_transactions** which if set to 0 disables usage of prepared transactions. We need to set this parameter to a non-zero value. The memory allocation, based on how the parameter is set, is [explained here](#). Oracle DB by default allows usage of prepared transactions.

XA transactions are not recommended by DBA due to:

- No way to identify a XA blocked transaction
- Vertica (powered by Postgres) -> Auto Vacuum didn't run for long period which lead to 24 hr outage (After completing the vacuum, restored the db).

### Chain Transaction Management

ChainedTransactionManager is about as plug-and-play of a solution as it gets. Once you get the configuration part done, it's simply a matter of telling your app via annotations at the method level, where to apply the new chained transaction handler.

### Why one DB in pre-prod and Staging

There would always be one database archive Aurora Postgres cluster in PreProd account for monolith as well as Audit. This will be used to archive data from any other non-prod databases (SYS, PDS and any other non-prod environments).

### Pause on Data Archival Track

#### Not much ROI because of below:

- 5TB (one time archival) out of ~27 TB total db size. We have storage limit of 128TB in Postgres.
- Complicate Oracle Exit migration journey.
  - It would cause lot of bloating on postgres because of heavy DMLs.
  - More frequent Vacuum runs on postgres database
- Heavy DMLs on source oracle will lead to lag/replication issues on ongoing DMS replication.
- Regular Data validation has to be in place and it would be more complicated.
- Once we finalize partitioning strategy for existing partitioned tables and other Non-partitioned tables (greater than 300G), it will lead to rework or another migration for this.

#### Use-cases:

- Restore back from archive db to production db
- Performance overhead ?
- Any jobs/reports to run out of this database for archived companies/data?

#### Prerequisites/Considerations

- Use Postgres schema generator to create schema and tables ?
- Data Type Mapping ?
- DB Archival schema will have tables, indexes, constraints (including FK).
- DB Archival schema will not have converted/Postgres Compatible PL/SQL code (Functions, Packages, Procedures), triggers, View ?

### Rollback Strategy

TBAdded

### Clients of PSP DB

Components/process/Services need to be analysed which may required company to be restored back to main DB.

- DG request
- Workforce requests
- Agent driven Restore request scenarios (entitlement activation, migration etc.)
- dBill system (AMO message, ERS, siebel etc.)
- Equifax
- AMS
- Upstream and Downstream Services List
- Case360 (W2 storage)

# References

[QBOE Data Purging](#)

IOP DB Archiving & Purging : [Graveyard](#)

Company Migrator tool : <https://github.intuit.com/payroll-services-platform/company-migrator>